Mern App

Cloud-Native MERN Microservices Deployment on AWS

Docker • Amazon ECR • Jenkins CI/CD • Amazon EKS • Helm • CloudWatch • ChatOps

Main Repository	https://github.com/UnpredictablePrashant/StreamingApp.git
Architecture	Microservices (Auth, Streaming, Admin, Chat) + React Frontend
Cloud Platform	Amazon Web Services (ECR, EKS, CloudWatch, SNS, S3)
CI/CD	Jenkins on EC2 → Build & Push Docker Images → Helm Deploy to EKS
Database	MongoDB (shared across microservices)
Object Storage	Amazon S3 (video assets + adaptive streaming)

This document describes the complete system architecture, containerization strategy, CI/CD pipeline, Kubernetes deployment, monitoring, and ChatOps integration for the Mern App collaborative video platform.

1. Project Overview

Mern App is a collaborative video streaming platform built with a modern microservices architecture. Users can stream premium video content, host live watch parties with real-time chat, and manage the content catalogue through a dedicated admin portal. The application is containerized and designed for production deployment on Amazon EKS.

1.1 Application Components

Service	Port	Responsibility
frontend	3000	React SPA – cinematic UI, player, chat overlay, role-aware routes
authService	3001	User registration, login, JWT issuance & validation
streamingService	3002	Video catalogue, public APIs, S3 signed playback endpoints
adminService	3003	Admin portal APIs – uploads, metadata, featured curation
chatService	3004	WebSocket (Socket.IO) + REST for live watch-party chat
mongo	27017	Shared MongoDB instance for all microservices

1.2 Key Features

- S3-backed adaptive streaming with secure signed uploads for administrators.
- Dedicated admin microservice for video ingestion, metadata management, and featured content.
- Real-time chat overlay in the video player (Socket.IO + persistent message history).
- Modern React experience with cinematic hero sections, dynamic carousels, and responsive design.
- Role-aware access control across frontend routes and backend microservices.

2. System Architecture

The diagram below illustrates the end-to-end architecture: users access the React frontend through an Application Load Balancer / Ingress controller. Traffic is routed to the appropriate microservice. All services share a MongoDB database and interact with Amazon S3 for video assets. The CI/CD pipeline is powered by Jenkins, which builds Docker images and pushes them to Amazon ECR. Helm charts deploy and manage the workloads on Amazon EKS. Observability is provided by Amazon CloudWatch with optional SNS-based ChatOps notifications.

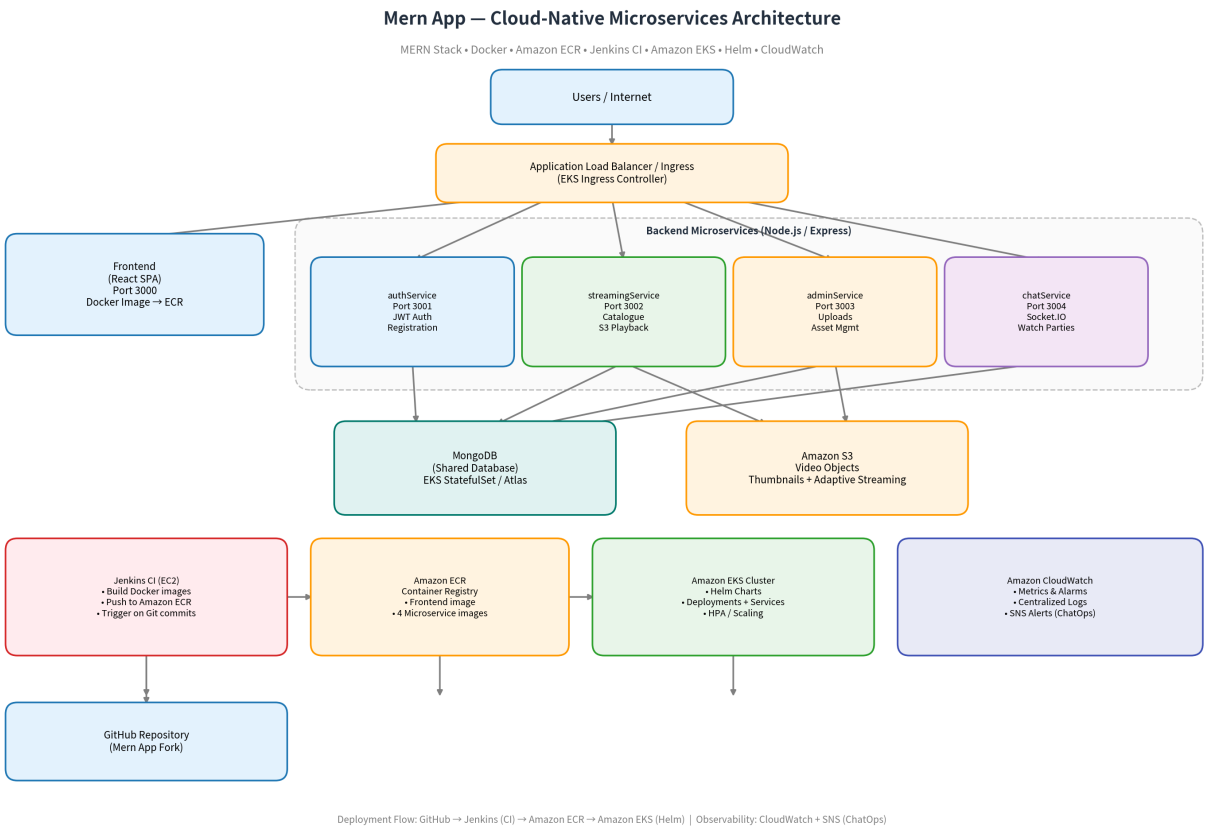


Figure 1: High-level architecture of Mern App on AWS

3. Version Control with Git

The starting point is the official repository. Every team member should maintain their own fork and keep it synchronized with the upstream repository.

Steps

1. Fork the main repository into your GitHub account.
2. Clone your fork locally and add the upstream remote:

```
git clone https://github.com/<your-username>/StreamingApp.git
cd Mern App
git remote add upstream https://github.com/UnpredictablePrashant/StreamingApp.git
```

3. Periodically sync your fork:

```
git fetch upstream
git checkout main
git merge upstream/main
git push origin main
```

4. Containerization (Docker)

Each component (frontend + four backend microservices) is packaged into its own Docker image. A multi-stage build is recommended for the React frontend to keep the final image small.

4.1 Example Dockerfile (Node.js microservice)

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
```

```
RUN npm ci --only=production
COPY . .
EXPOSE 3001
CMD ["node", "index.js"]
```

4.2 Frontend multi-stage Dockerfile (recommended)

```
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Local validation can be performed with the provided docker-compose.yml, which starts MongoDB together with all four microservices.

5. Push Docker Images to Amazon ECR

Create one ECR repository per image (frontend, authService, streamingService, adminService, chatService). Authenticate Docker to ECR, build, tag, and push.

```
# Create repository (example)
aws ecr create-repository --repository-name mernapp/auth-service --region ap-south-1

# Authenticate
aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin
<account>.dkr.ecr.ap-south-1.amazonaws.com

# Build, tag, push
docker build -t mernapp/auth-service ./backend/authService
docker tag mernapp/auth-service:latest
<account>.dkr.ecr.ap-south-1.amazonaws.com/mernapp/auth-service:latest
docker push <account>.dkr.ecr.ap-south-1.amazonaws.com/mernapp/auth-service:latest
```

6. AWS Environment Setup

Install and configure the AWS CLI with an IAM user that has sufficient permissions for ECR, EKS, CloudWatch, S3, and SNS.

```
aws configure
# Provide Access Key ID, Secret Access Key, default region (e.g. ap-south-1), and output format
```

7. Continuous Integration with Jenkins

Jenkins is installed on an Amazon EC2 instance. Required plugins typically include Docker, Amazon ECR, Pipeline, Git, and Credentials Binding. Store AWS credentials and GitHub credentials securely in the Jenkins credential store.

Pipeline Responsibilities

- Checkout source code from the GitHub fork on every push / PR.
- Build Docker images for frontend and all microservices.
- Tag images with commit SHA and/or semantic version.

- Authenticate to Amazon ECR and push the images.
- Optionally trigger a Helm upgrade on the EKS cluster.

High-level Jenkinsfile structure

```
pipeline {
  agent any
  stages {
    stage('Checkout') { steps { git ... } }
    stage('Build Images') { steps { sh 'docker build ...' } }
    stage('Push to ECR') { steps { sh 'docker push ...' } }
    stage('Deploy') { steps { sh 'helm upgrade --install ...' } }
  }
}
```

8. Kubernetes Deployment on Amazon EKS

Create an EKS cluster using eksctl (or the AWS console / Terraform). Package the entire application as Helm charts so that deployments, services, configmaps, secrets, and ingress resources can be versioned and upgraded together.

8.1 Create Cluster (example)

```
eksctl create cluster \
  --name mernapp-cluster \
  --region ap-south-1 \
  --nodegroup-name standard-workers \
  --node-type t3.medium \
  --nodes 3 \
  --nodes-min 2 \
  --nodes-max 5 \
  --managed
```

8.2 Helm Deployment

```
helm upgrade --install mernapp ./helm/mernapp \
  --namespace mernapp --create-namespace \
  --set image.tag=<git-sha> \
  --set mongodb.uri=... \
  --set s3.bucket=...
```

Recommended Kubernetes resources: Deployment + Service for each microservice, Ingress (ALB Ingress Controller or NGINX), HorizontalPodAutoscaler, ConfigMaps / Secrets for environment variables, and a StatefulSet or external managed MongoDB (e.g. Atlas).

9. Monitoring and Logging

Amazon CloudWatch provides metrics, logs, and alarms for the EKS cluster, EC2 instances (Jenkins), and application-level custom metrics. Container logs can be shipped to CloudWatch Logs via the Fluent Bit / CloudWatch agent or the AWS for Fluent Bit DaemonSet.

Suggested Alarms

- High CPU / memory utilization on worker nodes or individual pods.
- Elevated 5xx error rates from the ALB / Ingress.
- Pod restart count exceeding a threshold.
- Jenkins build failure notifications.

10. Bonus – ChatOps Integration

Create Amazon SNS topics for deployment success, deployment failure, and critical alarms. Subscribe a Slack workspace, Microsoft Teams channel, or Telegram bot to the topics so that the team receives real-time notifications.

```
aws sns create-topic --name mernapp-deploy-success
aws sns create-topic --name mernapp-deploy-failure
# Subscribe endpoints (email, Lambda that posts to Slack, etc.)
```

11. Documentation & Final Validation

All architecture diagrams, Helm values, Jenkinsfile, Dockerfiles, and operational runbooks should be committed to the GitHub repository. Final validation checklist:

- ■ Frontend is reachable via the public Ingress / ALB URL.
- ■ User registration and login succeed (authService).
- ■ Video catalogue loads and playback works (streamingService + S3).
- ■ Admin can upload a test video and thumbnail (adminService).
- ■ Real-time chat messages appear across multiple browser sessions (chatService).
- ■ Jenkins pipeline builds and pushes images on a new commit.
- ■ Helm upgrade successfully rolls out a new image tag.
- ■ CloudWatch shows healthy metrics and no critical alarms.
- ■ SNS notifications are received for a successful / failed deployment.

12. Submission Instructions

• Ensure the project is complete and tested end-to-end.
• Push all code, Helm charts, Jenkinsfile, and this documentation to your GitHub repository.
• Share the repository link in a text, Word, or PDF file.
• Upload the file containing the link via the Vlearn platform.

End of Document — Mern App Cloud Deployment Project